

Chapter 12: Decision Trees for Predictive Regression

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to explain how decision trees recursively partition data using split criteria that minimize prediction error within resulting groups

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to train regression trees using scikit-learn and visualize tree structure to trace prediction paths

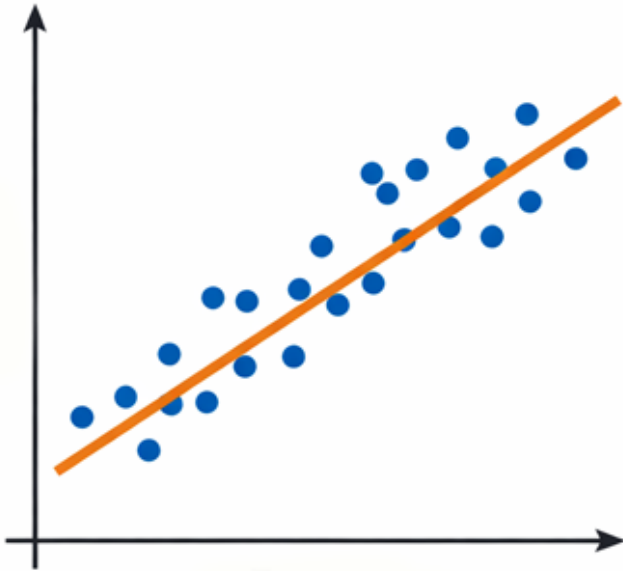
<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to compute and interpret impurity-based feature importance to identify the most influential predictors

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to apply regularization hyperparameters (max_depth, min_samples_split, min_samples_leaf) to control overfitting in decision trees

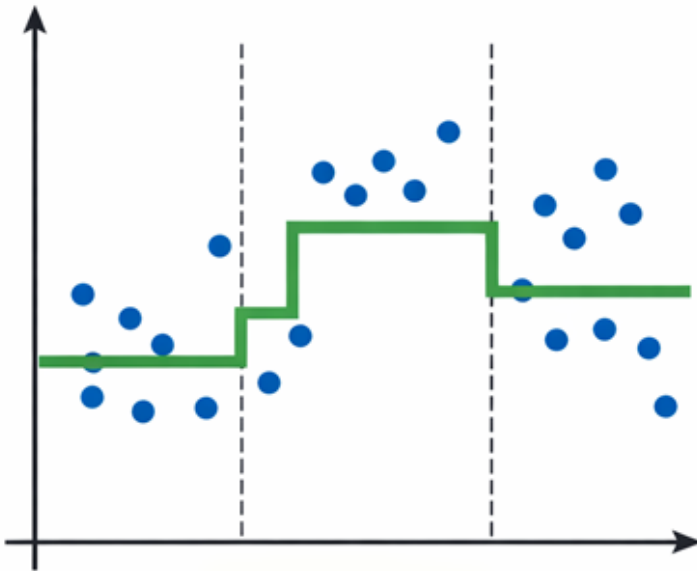
<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to compare decision trees versus linear regression on interpretability, nonlinearity handling, and assumption requirements

12.1 Introduction

Linear Regression



Decision Tree Regression



In the previous chapter, you learned how to build *predictive regression models* by focusing on generalization, error metrics, and feature selection

workflows. In this chapter, you will learn a new modeling algorithm: *decision trees*. Unlike linear regression, decision trees can naturally represent nonlinear patterns and interaction effects without requiring you to manually engineer many transformed features.

This chapter is intentionally about the *decision tree algorithm itself*, not about a specific prediction task such as regression or classification. A decision tree is a flexible modeling framework that can be used for both. In later chapters, you will apply decision trees to classification problems and compare them to other classification algorithms (including logistic regression).

A decision tree works by repeatedly splitting the data into smaller and more homogeneous groups. Each split is based on a simple question about a single feature, such as: “Is *age* greater than 45?” or “Is *smoker* equal to yes?” After enough splits, the model produces a prediction for each final group.

A helpful way to think about decision trees is that they learn a set of *if-then rules*. Each internal node asks a question, each branch represents a possible answer, and each leaf node stores a final prediction. In regression trees, the prediction is typically a numeric value (such as the average label value in that leaf). In classification trees, the prediction is typically a class label or a probability.

Decision trees are often introduced early in machine learning courses for three reasons. First, they are intuitive: the logic can be read and explained like a flowchart. Second, they handle mixed data types well, including numeric and categorical features. Third, they naturally capture nonlinear relationships and interactions by splitting the feature space into regions.

However, decision trees also come with tradeoffs. A single tree can overfit easily by memorizing small patterns in the training data. This means that, for prediction, you must be careful about controlling tree complexity and evaluating performance on held-out data. Later in the book, you will learn how tuning and ensemble methods (such as random forests and gradient boosting) reduce overfitting and often outperform a single tree.

In this chapter, you will focus on four foundational questions:

- What does a decision tree model look like, and how does it make predictions?
- How does a tree decide where to split the data at each step?
- How do we control tree complexity to reduce overfitting?
- How do we evaluate trees fairly using training/validation/test logic?

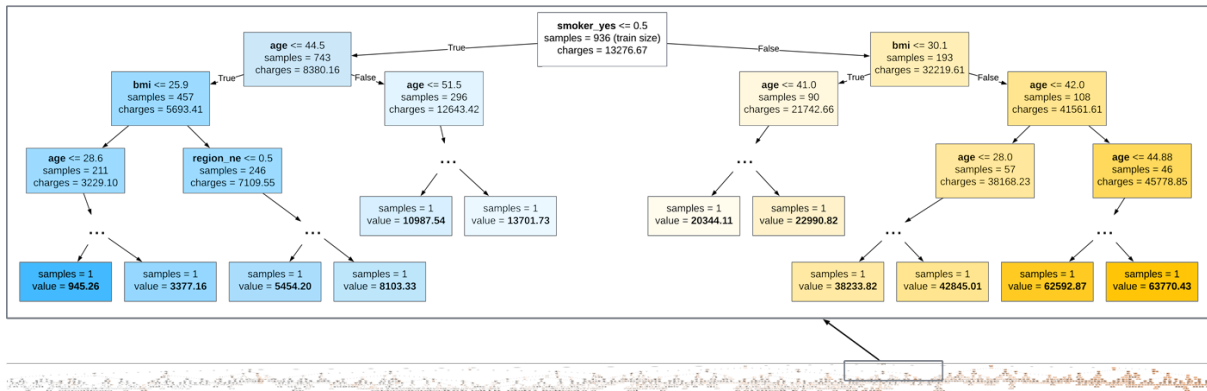
To maintain continuity, we will continue using familiar workflows from earlier chapters: separating predictors from the label, avoiding leakage, using train/test splits when prediction is the goal, and evaluating performance using appropriate metrics. The main difference is that you will now be fitting a model whose structure is not a single equation, but a collection of branching rules learned from the data.

Decision trees also provide a natural bridge between causal and predictive thinking. In a causal workflow, you typically care about interpretability and defensible assumptions. In a predictive workflow, you care about out-of-sample performance. Trees can be used in both settings, but they are most commonly used as predictive models because they can capture complex patterns that linear regression cannot.

Before building decision tree models in Python, you need to understand the basic structure of a tree and the logic behind how it is grown. In the next

section, you will learn how trees split data and how those splits create a set of decision rules.

12.2 How Trees Make Predictions



The figure above shows a trained decision tree regression model built using the insurance dataset, where the target variable is annual medical charges.

Each rectangle represents a node in the tree and displays three key pieces of information: the split rule, the number of training samples reaching that node, and the predicted charge value (the mean of those samples).

Key tree vocabulary

- **Node:** A point in the tree where a decision is made (internal node) or where a prediction is produced (leaf).
- **Root:** The first node at the top of the tree, where the first split occurs.
- **Split:** A rule that partitions the data into two groups (for example, smoker vs non-smoker).
- **Branch:** The path that data follows after a split.
- **Leaf:** A terminal node that produces the model's prediction.

- **Depth:** The number of splits from the root to a leaf.

What a regression tree predicts at a leaf

In a regression tree, each leaf corresponds to a subset of training rows, and the prediction stored in that leaf is the mean of the training labels in that subset.

For example, in the figure above, one leaf predicts approximately \$63,770.43 based on a very small subgroup of high-cost smokers, while another leaf predicts about \$945.26 for a low-cost non-smoker subgroup.

Conceptually, each leaf answers the question: “Among training cases that look like this, what outcome is typical?”

Following one path through the tree

The model begins at the root node with the rule **smoker_yes** $\leq$ **0.5**, separating non-smokers from smokers using all 936 training observations.

At this root, the average medical charge is approximately \$13,276.67, which would be the prediction if no further splits were made.

Suppose a new person is a non-smoker, so their data follows the left branch to the next node *age* $\leq$ 44.5, which contains 743 samples with an average charge of \$8,380.16.

If the person is younger than 44.5, the tree evaluates *bmi* $\leq$ 25.9 using 457 samples with mean charges of \$5,693.41.

Further splits on age or region continue narrowing the group until a leaf node is reached, at which point the stored mean becomes the prediction.

What happens for smokers

Smokers follow the right branch of the root, where predicted costs increase dramatically.

One smoker subgroup defined by $age \leq 41.0$ contains 90 samples with average charges of \$21,742.66.

Another branch with $bmi \leq 30.1$ contains 193 samples and predicts only \$3,221.61, showing how trees capture sharp nonlinear differences.

How the tree decides where to split

Training a tree means choosing which features to split on and which thresholds to use in order to improve predictive accuracy.

In regression, the most common criterion is to minimize within-group error so that observations within each leaf have similar outcomes.

Split quality in regression

A common measure of node quality is mean squared error relative to the node's mean prediction.

The tree evaluates many candidate splits and selects the one that reduces this error the most.

- Compute error in the current node.
- Try many possible splits.
- Compute weighted error after each split (i.e., the difference between the mean of charges for the node and summed difference between each actual

value of charges from that mean).

- Choose the split with the largest error reduction.

How trees handle numeric vs categorical features

Decision trees naturally handle both numeric and categorical inputs.

- **Numeric features:** split using thresholds such as $bmi \leq 30.1$.
- **Categorical features:** split by category membership such as smoker vs non-smoker.

Because trees split feature-by-feature, they do not require linearity or constant variance assumptions.

Why ≤ 0.5 for binary features?

You may notice that the tree uses split rules such as $smoker_yes \leq 0.5$ instead of a more intuitive condition like $smoker = no$.

This happens because many tree implementations internally represent binary categories as numeric values: 0 for “no” and 1 for “yes.”

A threshold of 0.5 simply separates these two values:

- $smoker_yes \leq 0.5 \rightarrow$ non-smokers (0)
- $smoker_yes > 0.5 \rightarrow$ smokers (1)

Although the rule is numeric, the conceptual meaning is still categorical. The tree is asking a simple yes/no question about smoking status.

Recursive partitioning

Trees are built using recursive partitioning, where each new split is applied separately to each subgroup created by earlier splits.

Early splits capture dominant effects such as smoking status, while later splits capture subtle patterns such as BMI differences among smokers.

Stopping rules and overfitting

If allowed to grow without limits, trees can memorize training data and overfit.

- Maximum depth.
- Minimum samples per split.
- Minimum samples per leaf.
- Minimum impurity decrease.

These parameters control the bias–variance tradeoff and are tuned using validation or cross-validation.

Table 12.1
Decision trees vs linear regression

Property	Decision trees	Linear regression
Interpretability	Very high; predictions follow explicit if–then rules and can be traced visually from root to leaf.	Moderate; coefficients are interpretable but combined effects are less intuitive.
Handling nonlinearity	Excellent; naturally captures thresholds and interactions.	Poor without manual feature engineering.
Handling interactions	Automatic via recursive splits.	Must be explicitly specified.

Property	Decision trees	Linear regression
Stability	Lower; small data changes can produce different tree structures.	Higher; coefficients change smoothly with data.
Overfitting risk	High for deep trees without regularization.	Lower when assumptions hold.
Assumptions	Few; no linearity or constant variance required.	Strong; linearity, independence, and homoscedasticity.

12.3 Training a Regression Tree

Now that you understand how regression trees make predictions and choose splits, we will train a decision tree model using the insurance dataset to predict medical charges.

Our goal is not yet to optimize performance, but to see how the abstract ideas from the previous section appear in a real fitted model.

Dataset and target

We will use the same insurance dataset from earlier chapters, with *charges* as the numeric target variable and all remaining columns as predictors.

Decision trees can directly handle both numeric and categorical predictors, but in practice most Python libraries expect categorical variables to be encoded numerically.

Train–test split

As in predictive regression, we divide the data into a training set and a test set. The training set is used to learn the tree structure, and the test set is reserved for final evaluation.

This separation is especially important for trees because deep trees can easily memorize training data.

Model choice

We use the CART-style regression tree implemented in *sklearn.tree.DecisionTreeRegressor*.

By default, this algorithm selects splits that minimize mean squared error inside each node.

Basic training workflow

The workflow consists of four steps:

- Prepare features and target.
- Encode categorical variables.
- Fit the tree on the training data.
- Evaluate predictions on the test set.

Python example

The code below demonstrates a minimal training pipeline using scikit-learn.



```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
```

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load data
df = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv")

y = df["charges"]
X = df.drop(columns=["charges"])

# One-hot encode categorical variables
X = pd.get_dummies(X, drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Train regression tree
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)

# Predictions
y_pred = tree.predict(X_test)

# Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)

# Output:
# MAE: 3195.1104733805973
# RMSE: 6515.129162967606
# R2: 0.7265877305258355

```

Interpreting the results

If you run this code with default settings, you will typically observe very low training error but noticeably worse test-set error.

This is a classic symptom of overfitting: the tree has grown deep enough to closely match the training data but does not generalize as well to new observations.

In the next section, we will visualize the fitted tree to see how deep it becomes and how many splits it creates.

Why we start with an untuned tree

Beginning with a fully flexible tree is useful for learning because it exposes the model's natural tendency to overfit.

Once this behavior is clear, we can introduce regularization techniques such as limiting tree depth and minimum leaf size to improve generalization.

This mirrors the earlier regression chapters, where we first fit unrestricted models before introducing validation and tuning.

Next, we will visualize the trained tree and connect its structure back to the concepts of nodes, splits, depth, and leaves.

12.4 Visualizing a Regression Tree

Decision trees are especially useful in business settings because you can **visualize** the model and interpret its logic as a sequence of human-readable rules.

In this section, you will train a regression tree using the insurance dataset and generate a tree diagram. In the next part of this section, you will interpret the diagram by tracing split rules and connecting leaves to predicted medical charges.

Generate a regression tree visualization in Python

This code trains a decision tree regression model to predict *charges*. Because the insurance dataset contains categorical variables (*sex*, *smoker*, *region*), we build a preprocessing pipeline that one-hot encodes categories and passes numeric values through unchanged.

The output of the pipeline is a numeric matrix suitable for scikit-learn modeling. We then visualize the trained tree and save the image so it can be embedded in the textbook.



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_absolute_error, mean_squared_error

# 1) Load data
df = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv")

y = df["charges"]
X = df.drop(columns=["charges"])

# Identify column types
num_cols = X.select_dtypes(include=["int64",
"float64"]).columns.tolist()
cat_cols = X.select_dtypes(include=["object"]).columns.tolist()

# 2) Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# 3) Preprocessing (one-hot encode categoricals; pass numeric through)
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
        ("num", "passthrough", num_cols)
    ],
    remainder="drop"
)

# 4) Model (choose a depth so the diagram is readable)
tree_model = DecisionTreeRegressor(
    max_depth=3,
    min_samples_leaf=20,
    random_state=42
)

model = Pipeline(steps=[
    ("prep", preprocessor),
    ("tree", tree_model)
])

# Fit
model.fit(X_train, y_train)
```

```

# 5) Quick evaluation (so we have context for the tree)
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Test MAE: {mae:,.2f}")
print(f"Test RMSE: {rmse:,.2f}")

# 6) Build feature names for the plotted tree
# Get the fitted OneHotEncoder and build readable feature names
ohe = model.named_steps["prep"].named_transformers_["cat"]
ohe_feature_names = ohe.get_feature_names_out(cat_cols).tolist()
feature_names = ohe_feature_names + num_cols

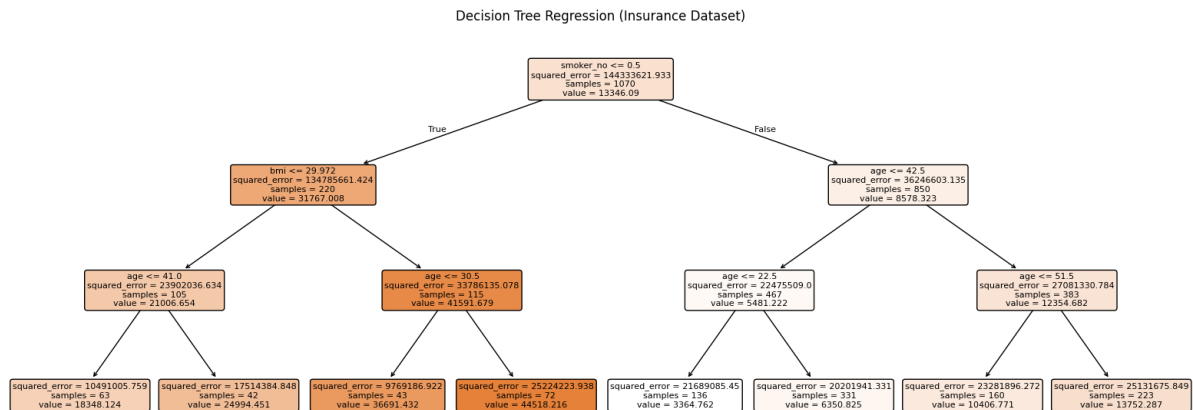
# 7) Plot and save the tree image
plt.figure(figsize=(16, 6)) # widescreen
plot_tree(
    model.named_steps["tree"],
    feature_names=feature_names,
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Decision Tree Regression (Insurance Dataset)")
plt.tight_layout()

# Save to a file you can move into your book images folder
plt.savefig("decision_tree_insurance.png", dpi=1200)
plt.show()

# Output:
# Test MAE:  2,865.64
# Test RMSE: 4,776.26

```

After you run the code, you should see a tree diagram and a saved image file named *decision_tree_insurance.png*. The diagram shows each split rule, how many training samples reached each node, and the predicted value (mean *charges*) stored at each leaf.



Next, you will interpret the tree by tracing paths from the root to leaves and explaining what the split conditions mean in business terms (for example, how smoking status and BMI thresholds segment expected medical charges).

The figure above shows the trained regression tree for the insurance dataset with a maximum depth of 3. Each internal node displays the split rule, the number of training samples that reached that node, and the mean medical charge stored at that node. Each leaf represents the model's final prediction for observations that follow that path.

The root node splits on **smoker_no ≤ 0.5** , which separates smokers from non-smokers. This split alone divides the dataset into two large groups with dramatically different average charges: approximately \$13,346 overall, but only \$8,578 on the smoker branch and much higher values on the non-smoker branch.

On the left side (non-smokers), the next split occurs on **bmi ≤ 29.97** . Individuals with lower BMI move left toward a group with an average charge near \$21,007, while higher-BMI non-smokers move right toward a group averaging over \$41,592.

For example, the leaf reached by non-smokers with BMI greater than 29.97 and age greater than 30.5 predicts an average medical charge of approximately \$44,518. In contrast, younger non-smokers with lower BMI fall into a leaf predicting roughly \$18,348.

On the right side of the root (smokers), the tree first splits on **age ≤ 42.5** . Younger smokers are predicted to have much lower charges (around \$5,481 on average), while older smokers move into higher-cost groups averaging above \$12,354.

Following the rightmost branch further, smokers older than 51.5 are predicted to incur average charges exceeding \$13,752, while those between 42.5 and 51.5 average around \$10,406.

How to read a regression tree prediction

To make a prediction for a new individual, the model starts at the root and evaluates each split condition in order. The observation follows the left or right branch depending on whether it satisfies the condition, until it reaches a leaf node.

The numeric value shown at the leaf (labeled *value*) is the model's predicted medical charge. This value is simply the mean of all training observations that followed the same path during training.

Why this structure works well for nonlinear relationships

This visualization highlights how decision trees naturally model **nonlinear effects** and **interactions**. The effect of BMI depends on smoking status, and the effect of age depends on both BMI and smoking. In a linear regression model, these relationships would require manually specified interaction terms.

Instead, the tree discovers these interactions automatically by recursively partitioning the feature space into regions with similar outcomes.

Summary

The tree diagram turns the model into a set of transparent decision rules: first split by smoking status, then by BMI or age, and finally by finer age

thresholds. Each path represents a subgroup with a distinct expected medical cost.

This combination of predictive power and interpretability is one of the main reasons decision trees are widely used in applied machine learning and serve as the foundation for more advanced models such as random forests and gradient-boosted trees.

12.5 Tree-based feature importance

After training a regression tree, a natural question is: **which features mattered most** for reducing prediction error? Decision trees provide a built-in answer called *impurity-based feature importance* (sometimes called *MDI*, for mean decrease in impurity).

What “feature importance” means in a tree

A regression tree is built by repeatedly choosing splits that reduce within-node error. In scikit-learn regression trees, that error is measured using **mean squared error** (MSE), which appears in the visualization as *squared_error*.

Every time the tree splits on a feature, it reduces the parent node’s error by some amount. The tree’s importance score for a feature is based on the total amount of error reduction attributed to that feature across all splits where it was used.

Conceptually, the importance for a feature is:

- At each split, compute how much the split reduced error (parent MSE minus the weighted child-node MSE).
- Assign that reduction to the feature used at the split.

- Sum reductions across the entire tree for each feature.
- Normalize the totals so the importances add up to 1.0.

Because this approach measures how much each feature helped reduce training-set error inside the tree structure, it is fast and easy to compute. It also pairs well with tree visualizations: if a feature appears near the top of the tree, it often has high importance because it drives large early error reductions.

Notice that importance scores are always **non-negative** and measure only the *magnitude* of a feature’s contribution to error reduction—not the *direction* of its effect on the prediction. A high importance score tells you that the feature was heavily used for splitting, but it does not tell you whether larger values of that feature push the prediction up or down. This contrasts with linear regression coefficients, which carry a sign indicating direction. When you need to understand how a specific feature influences the target, examine the tree’s split structure or use partial-dependence techniques introduced in later chapters.

Important limitations of impurity-based importance

Impurity-based importance is useful, but it is not perfect evidence of “true” predictive value. In particular, importance scores can be biased toward features that have many potential split points (continuous variables often have more split opportunities than low-cardinality categorical variables) and toward correlated predictors where the tree can substitute one for another.

For these reasons, treat impurity-based importance as a helpful **first look**. In later chapters, you will learn more robust approaches such as *Permutation Feature Importance (PFI)* and cross-validation-based evaluation, which

provide stronger evidence about how much a feature contributes to out-of-sample accuracy.

Computing and plotting feature importance in Python

The code below (1) loads the insurance dataset, (2) prepares the predictors using dummy coding for categorical variables, (3) trains a regression tree, and (4) creates a bar chart of impurity-based feature importance scores.

This example intentionally keeps the workflow simple by using *pandas.get_dummies()*. In later sections, you will see how to implement similar preprocessing using sklearn pipelines for safer deployment workflows.



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

df = pd.read_csv(""/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv"")

y = df["charges"]
X = df.drop(columns=["charges"])

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

tree = DecisionTreeRegressor(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

importances = pd.Series(tree.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=False)

top_k = 12
top_imp = importances.head(top_k)[: -1]

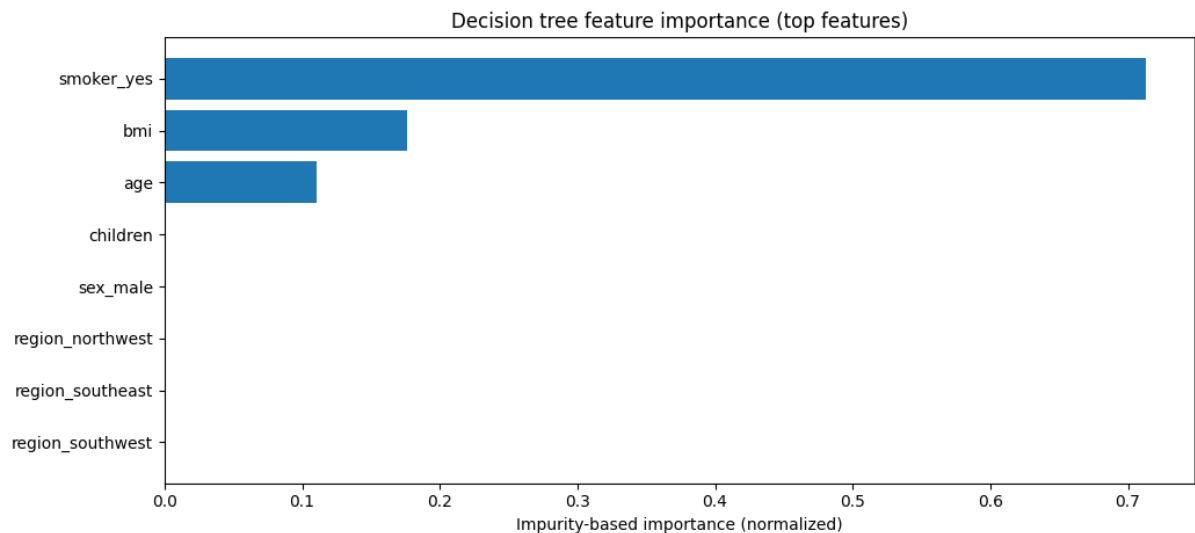
plt.figure(figsize=(10.5, 4.8))
plt.barh(top_imp.index, top_imp.values)
plt.xlabel(""Impurity-based importance (normalized)"")
```

```
plt.title(""Decision tree feature importance (top features)"")
plt.tight_layout()
plt.savefig(""tree_feature_importance.png"", dpi=200,
bbox_inches="tight",)
plt.show()

print(importances.head(12))

# Output:
# smoker_yes          0.712802
# bmi                 0.176411
# age                 0.110787
# children            0.000000
# sex_male            0.000000
# region_northwest    0.000000
# region_southeast    0.000000
# region_southwest    0.000000
# dtype: float64
```

The printed output shows the same information as the chart, but the chart is often easier to interpret. Features with higher bars contributed more total error reduction across the tree's splits.



How to interpret the chart

In the chart above, *smoker_yes* dominates the importance scores, accounting for the majority of total impurity reduction in the tree. This is consistent with what we observed in the visualization of the tree itself: the root split is based

on smoking status, meaning this feature produces the largest single reduction in prediction error.

The next most important features are *bmi* and *age*. These variables appear near the top of the tree and are used to further separate high-cost and low-cost subgroups within smokers and non-smokers. Features that appear earlier in the tree generally receive higher importance because they influence large portions of the data.

Notice that the remaining features (*children*, *sex_male*, and the regional indicators) have importance values of exactly zero. This does not mean they are useless predictors in general. It simply means that, in this particular model, the tree never used them for any split.

This happened because we intentionally limited the tree to a maximum depth of three using the *max_depth* hyperparameter in the prior section. With such a shallow tree, the algorithm can only make a small number of splits, so it selects the few features that reduce error the most early on and ignores the rest.

If the tree were allowed to grow deeper, additional features would likely appear in lower-level splits and would receive nonzero importance scores. We will study this tradeoff in detail later in this chapter when we discuss tree depth, regularization, and how hyperparameters control overfitting and model stability.

Why tree depth affects feature importance

Impurity-based feature importance depends entirely on which splits the tree is allowed to make. When a tree is shallow (for example, *max_depth* = 3), only a few features can appear in the model, so importance becomes concentrated in those early splits.

As tree depth increases, more features have opportunities to appear in lower branches. This usually spreads importance across a larger set of variables and reduces the dominance of any single feature.

In short: shallow trees produce *concentrated importance*, while deeper trees produce *distributed importance*. Neither is inherently better—the choice reflects a tradeoff between simplicity, stability, and predictive power.

Finally, remember that impurity-based importance reflects how useful a feature was for prediction inside this specific trained model. It does not imply causation, and it can change substantially if the tree structure or hyperparameters are changed.

Those who are more mathematically inclined may appreciate understanding a little more detail below:

How impurity reduction becomes feature importance (optional)

Internally, a regression tree measures the quality of a split using the reduction in mean squared error (MSE) produced by that split.

For a node containing n samples with target values $y_1, \dots, y_n$ and mean $\hat{y}$, the node impurity is:

$$MSE(node) = (1 / n) \cdot \sum (y_i - \hat{y})^2$$

When a candidate split divides the data into a left child and right child, the impurity after the split is the weighted average:

$$MSE_{after} = (n_L/n) \cdot MSE_L + (n_R/n) \cdot MSE_R$$

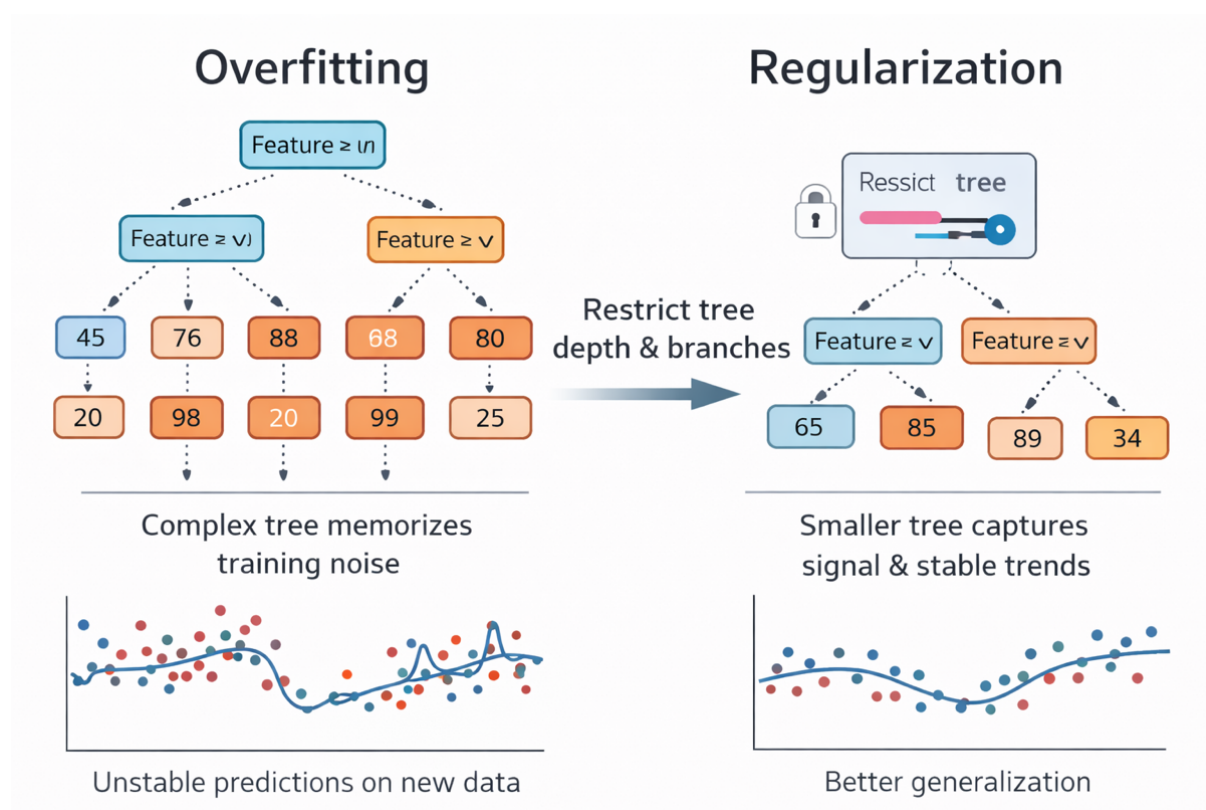
The impurity reduction for that split is:

$$\Delta = MSE_{before} - MSE_{after}$$

Every time a feature is used in a split, its Δ value is added to that feature's total importance score. After training, all feature totals are normalized so they sum to 1.0.

This explains why features used early and often (such as *smoker_yes* in our tree) dominate the importance chart: they repeatedly produce large impurity reductions over many samples.

12.6 Overfitting and Regularization



Decision trees are powerful because they can adapt to complex, nonlinear patterns. However, this flexibility also makes them especially prone to **overfitting**.

A tree overfits when it learns the noise and idiosyncrasies of the training data rather than the underlying signal. The result is excellent training performance but poor generalization to new observations.

Why regression trees overfit easily

Unlike linear models, trees can continue splitting until each leaf contains only a few observations (or even a single observation). When this happens, the model effectively memorizes the training set.

In a regression setting, this means many leaves will predict values extremely close to individual training points, producing near-zero training error but unstable predictions for new data.

Overfitting in trees typically appears as:

- Very deep trees with many splits.
- Leaves containing very few samples.
- Large gaps between training error and validation/test error.

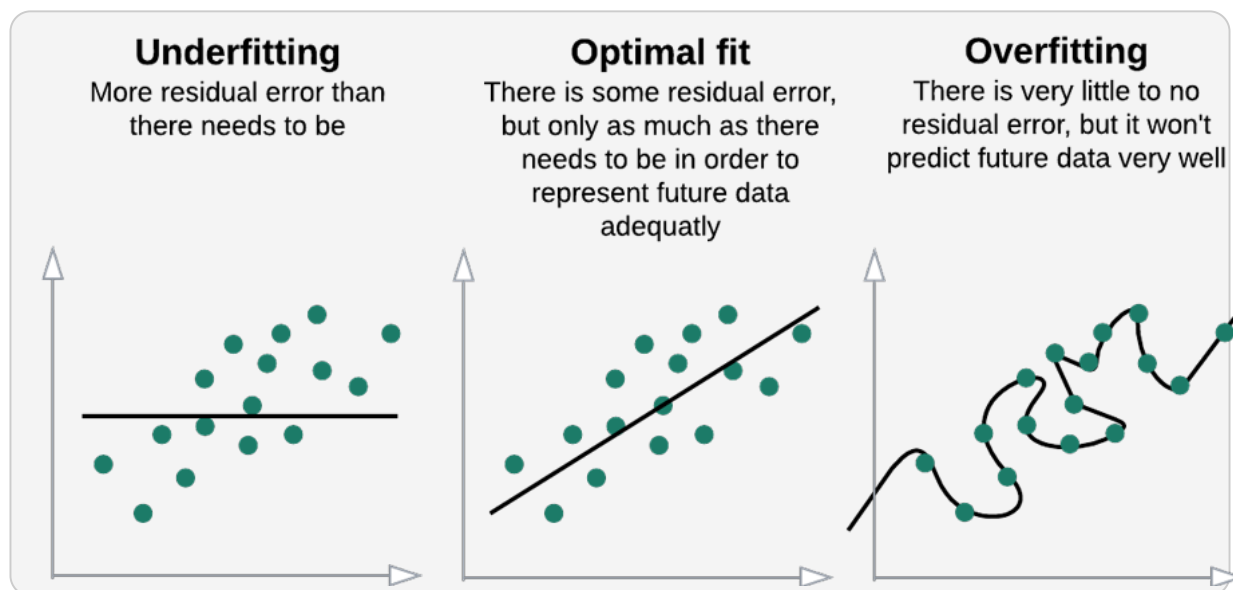
The bias–variance perspective

Tree complexity directly controls the bias–variance tradeoff.

Shallow trees have **high bias**: they are too simple to capture real structure. Deep trees have **high variance**: they fit the training data extremely well but change dramatically when the data changes slightly.

Good predictive performance lies between these extremes, where the tree is complex enough to capture signal but constrained enough to remain stable.

The figure below makes this tradeoff concrete using a simple regression example. On the left, the fitted line is nearly flat—it ignores the upward trend in the data entirely. This is **underfitting**: the model is too simple, leaving far more residual error than necessary. In the center, the line follows the overall trend without chasing every point. This is the **optimal fit**: some residual error remains, but only as much as needed to represent future data well. On the right, the curve twists through nearly every observation, producing almost no training error. This is **overfitting**: the model has memorized the noise in the training data and will predict poorly on new observations.



In the depth-sweep experiment that follows, you will see this same progression play out numerically. As tree depth increases, training RMSE drops steadily—the model moves from left to right in the figure. Test RMSE, however, improves only up to a point and then begins to rise, signaling the transition from optimal fit to overfitting.

A simple experiment: visualizing overfitting with tree depth

The code below generates a small synthetic regression dataset, trains multiple trees with increasing depth, and plots training and test RMSE as a function of *max_depth*. This produces the classic overfitting curve discussed above.



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

np.random.seed(42)

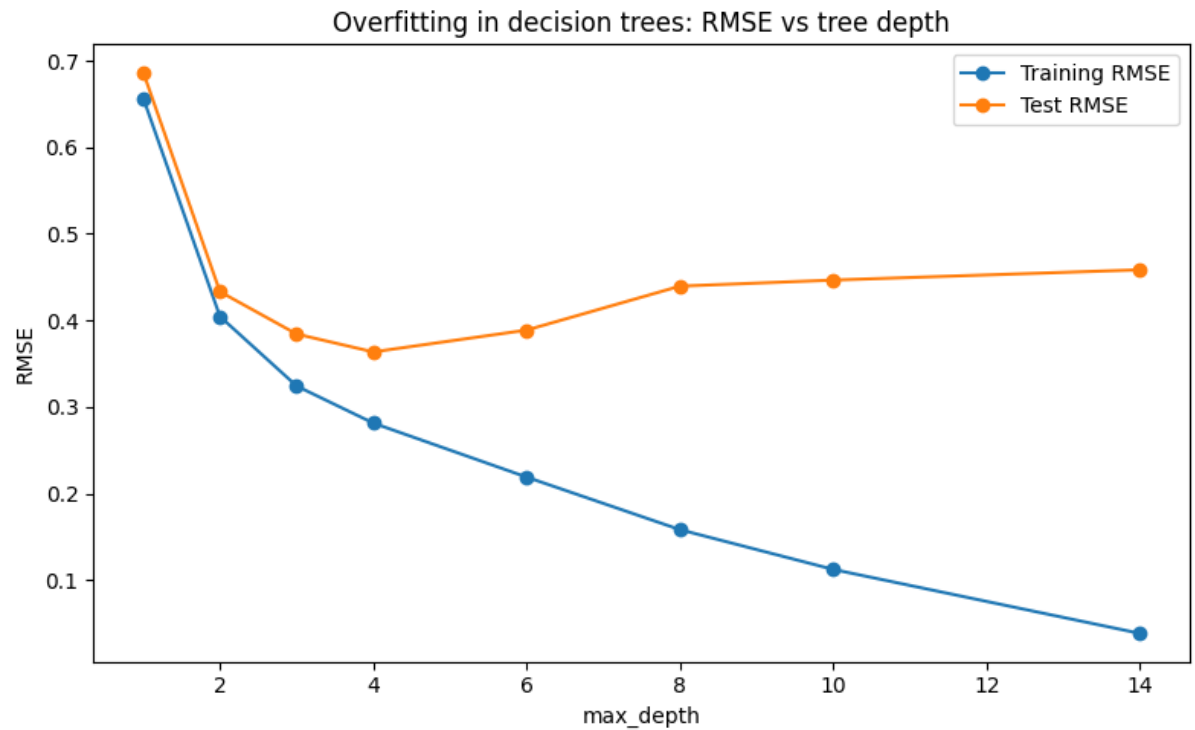
n = 400
X = np.linspace(0, 10, n).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.3, size=n)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

depths = [1, 2, 3, 4, 6, 8, 10, 14]
train_rmse = []
test_rmse = []

for d in depths:
    model = DecisionTreeRegressor(max_depth=d, random_state=42)
    model.fit(X_train, y_train)
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)
    train_rmse.append(np.sqrt(mean_squared_error(y_train, y_train_pred)))
    test_rmse.append(np.sqrt(mean_squared_error(y_test, y_test_pred)))

plt.figure(figsize=(8, 5))
plt.plot(depths, train_rmse, marker='o', label='Training RMSE')
plt.plot(depths, test_rmse, marker='o', label='Test RMSE')
plt.xlabel('max_depth')
plt.ylabel('RMSE')
plt.title('Overfitting in decision trees: RMSE vs tree depth')
plt.legend()
plt.tight_layout()
plt.show()
```



How to interpret the overfitting chart

The horizontal axis shows tree depth (model complexity), and the vertical axis shows prediction error (RMSE). Two curves are plotted: one for training error and one for test error.

At small depths, both curves are high and close together. This indicates **underfitting**: the tree is too simple to capture the true pattern in the data.

As depth increases, both training and test RMSE decrease. This means the model is learning meaningful structure that generalizes to new data.

Eventually, the training RMSE continues to drop sharply while the test RMSE flattens or decreases much more slowly. The gap between the curves begins to widen.

This widening gap is the visual signature of **overfitting**. The tree is improving its ability to memorize the training data but is no longer gaining much ability to predict new observations.

In practice, we usually select a depth near the bottom of the test-error curve, before the gap between training and test error becomes large.

What regularization means for trees

In decision trees, **regularization** does not mean shrinking coefficients (as in linear regression). Instead, it means *restricting the structure of the tree itself*.

Rather than letting the tree grow freely, we impose rules that limit how complex it can become.

Common regularization strategies

- **Maximum depth:** limits how many splits are allowed from root to leaf.
- **Minimum samples per split:** prevents splitting nodes that contain too few observations.
- **Minimum samples per leaf:** ensures that each leaf represents a reasonably sized group.
- **Minimum impurity decrease:** requires that each split reduce error by a meaningful amount.

All of these controls reduce variance by preventing the tree from fitting extremely specific patterns that are unlikely to repeat in new data.

Connection to feature importance

Regularization also affects feature importance. As you saw in the previous section, a shallow tree concentrates importance in a small number of features, while deeper trees distribute importance across many variables.

This means feature importance values are not purely properties of the dataset; they also reflect modeling choices about tree complexity.

Practical takeaway

Unrestricted trees almost always overfit in regression problems.

Controlling tree complexity is therefore not optional—it is essential for building models that generalize well.

In the next section, we will examine the specific hyperparameters that implement these controls in Python and how to tune them systematically.

12.7 Hyperparameters in Regression Trees

Unlike linear regression, a decision tree does not estimate a fixed set of coefficients. Instead, its structure is controlled by a set of user-defined **hyperparameters** that determine how deep the tree can grow, how many splits it is allowed to make, and how small each group of observations can become.

These hyperparameters directly control the model's complexity and therefore play a central role in the bias–variance tradeoff: shallow trees tend to underfit, while very deep trees tend to overfit.

Core hyperparameters for regression trees

The most important hyperparameters in practice are listed below.

Table 12.2

Common decision tree hyperparameters (regression)

Hyperparameter	What it controls	Effect when increased
max_depth	Maximum number of splits from root to leaf	More complex tree, lower bias, higher variance
min_samples_split	Minimum samples required to split a node	Fewer splits, simpler tree, higher bias
min_samples_leaf	Minimum samples required in each leaf	Smoother predictions, greater stability
max_features	Number of features considered at each split	More randomness, potentially less overfitting
min_impurity_decrease	Minimum error reduction required for a split	Prevents weak, low-impact splits

Why hyperparameters matter for prediction

Each split in a tree is chosen to reduce training-set error, but unrestricted splitting allows the model to chase small random patterns in the data.

Hyperparameters act as **structural regularization**: instead of shrinking coefficients (as in ridge regression), we restrict how detailed the tree is allowed to become.

In this section, we will train several trees with different hyperparameter combinations and compare their predictive performance. Because we care about generalization, we compare models using the same train/test split and compute test-set *MAE* and *RMSE* for each tree.

Training multiple trees and comparing model fit

The code below fits several regression trees on the insurance dataset using different values of *max_depth* and *min_samples_leaf*. It then evaluates each model on the test set and stores the results in a comparison table.



```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv")

y = df["charges"]
X = df.drop(columns=["charges"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

num_cols = X_train.select_dtypes(include=["int64",
"float64"]).columns.tolist()
cat_cols = X_train.select_dtypes(include=["object"]).columns.tolist()

preprocess = ColumnTransformer(
    transformers=[
        ("num", "passthrough", num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore",
drop="first"), cat_cols)
    ],
    remainder="drop"
)

param_grid = [
    {"max_depth": 2, "min_samples_leaf": 20},
    {"max_depth": 3, "min_samples_leaf": 20},
    {"max_depth": 4, "min_samples_leaf": 20},
    {"max_depth": 5, "min_samples_leaf": 20},
    {"max_depth": 3, "min_samples_leaf": 5},
    {"max_depth": 3, "min_samples_leaf": 50}
]

rows = []

for params in param_grid:
    tree = DecisionTreeRegressor(
        max_depth=params["max_depth"],
```

```

        min_samples_leaf=params[&quot;min_samples_leaf&quot;],
        random_state=42
    )

    model = Pipeline(steps=[
        (&quot;prep&quot;, preprocess),
        (&quot;tree&quot;, tree)
    ])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    rows.append({
        &quot;max_depth&quot;: params[&quot;max_depth&quot;],
        &quot;min_samples_leaf&quot;: params[&quot;min_samples_leaf&quot;],
        &quot;test_mae&quot;: mae,
        &quot;test_rmse&quot;: rmse,
        &quot;test_r2&quot;: r2
    })

results =
pd.DataFrame(rows).sort_values(by=&quot;test_rmse&quot;).reset_index(drop=True)
display(results)

```

	max_depth	min_samples_leaf	test_mae	test_rmse	test_r2
0	5	20	2616.925294	4506.035280	0.869214
1	3	50	2688.045462	4563.382047	0.865864
2	4	20	2695.495597	4576.182172	0.865110
3	3	20	2865.637810	4776.261075	0.853057
4	3	5	2865.637810	4776.261075	0.853057
5	2	20	3299.973030	5106.058522	0.832064

Run the code and examine the results table. In the next part of this section, we will interpret the tradeoffs you observe across models and connect them directly to overfitting, underfitting, and how hyperparameters shape tree structure.

Interpreting the Hyperparameter Comparison Results

Table results from the previous code block show clear differences in predictive performance across tree configurations. The model with **max_depth = 5** and **min_samples_leaf = 20** achieved the lowest test-set RMSE (4506.04) and MAE (2616.93), along with the highest R^2 (0.8692) among the candidates.

This configuration allows the tree to grow deeper than the others, capturing more structure in the data, while still enforcing a moderate minimum leaf size to prevent the model from fitting extremely small, unstable groups of observations.

At the other extreme, the shallowest tree (*max_depth* = 2) performs substantially worse, with RMSE above 5100 and noticeably lower R^2 . This is a classic example of **underfitting**: the model is too simple to capture the nonlinear patterns present in medical charges.

Several intermediate configurations (for example, *max_depth* = 3 or 4) fall between these two extremes. They reduce error relative to the shallow tree but do not perform as well as the deeper model with controlled leaf size.

How to choose hyperparameters in practice

In small examples like this one, you can manually compare a handful of reasonable parameter combinations and select the model that minimizes validation or test error while avoiding unnecessary complexity.

A practical workflow for students at this stage is:

- Start with a shallow tree (for example, *max_depth* = 2 or 3) to establish a baseline.

- Gradually increase *max_depth* while keeping *min_samples_leaf* reasonably large.
- Track test or validation RMSE and MAE to see where improvements level off.
- Choose the simplest model that achieves near-minimum error.

This balances two goals: strong predictive accuracy and model stability. Deeper trees often continue to reduce training error, but beyond a certain point they tend to generalize worse to new data.

Looking ahead: automated hyperparameter tuning

Manually testing combinations is useful for learning, but it does not scale well as the number of hyperparameters increases.

In later chapters, you will learn systematic approaches such as **grid search**, **random search**, and **cross-validation**, which automate this process and provide more reliable estimates of generalization performance.

For now, the key takeaway is that decision tree performance is highly sensitive to structural choices. Hyperparameters are not minor implementation details—they define the effective complexity of the model and largely determine whether a tree underfits, overfits, or generalizes well.

12.8 Strengths and Weaknesses

Decision trees offer a powerful alternative to linear regression, especially when relationships are nonlinear or complex. However, their flexibility also introduces important limitations. Understanding both sides is essential for choosing the right model in practice.

Strengths

- **No linearity assumption:** Trees do not assume a linear relationship between features and the outcome. They naturally capture thresholds, step changes, and nonlinear effects.
- **Automatic interaction modeling:** Trees model feature interactions automatically through their hierarchical splits, without requiring explicit interaction terms.
- **Handles mixed data types:** Numeric and categorical variables can be used together without extensive preprocessing or dummy-variable design.
- **Interpretability:** Small or moderately sized trees can be visualized and interpreted as a sequence of decision rules that are easy to explain to non-technical stakeholders.
- **Robust to outliers:** Because predictions are based on averages within leaves, extreme values tend to influence only local regions of the tree.

Weaknesses

- **High risk of overfitting:** Deep trees can memorize the training data, producing very low training error but poor performance on new data if not properly regularized.
- **Instability:** Small changes in the training data can lead to very different trees, splits, and feature importance rankings.
- **Lower predictive accuracy than ensembles:** A single tree is often outperformed by ensemble methods such as random forests and gradient boosting.
- **Piecewise-constant predictions:** Regression trees produce step-like prediction surfaces rather than smooth curves, which may be unrealistic

for some problems.

- **Sensitive to hyperparameters:** Performance depends heavily on choices such as maximum depth and minimum samples per leaf, which must be tuned carefully.

Trees versus linear regression

Regression trees and linear regression represent two very different modeling philosophies. Linear regression imposes strong structural assumptions in exchange for stability and interpretability, while trees sacrifice smoothness and stability in exchange for flexibility.

In problems where relationships are approximately linear and data are limited, linear regression often performs surprisingly well and is easier to trust for causal interpretation. In contrast, trees are better suited for complex predictive tasks with nonlinear patterns and interactions.

In modern predictive modeling, trees are rarely used alone. Instead, they serve as the foundation for powerful ensemble methods that reduce overfitting while preserving flexibility. We will move onto ensemble methods in a later chapter. For now, it may be useful to simply compare decision trees to linear regression.

Table 12.3
Decision Trees vs. Linear Regression

Aspect	Linear Regression	Decision Tree (Regression)
Relationship shape	Assumes linear effects	Captures nonlinear effects naturally
Feature interactions	Must be specified manually	Learned automatically via splits

Aspect	Linear Regression	Decision Tree (Regression)
Statistical assumptions	Linearity, independence, homoscedasticity, normality (for inference)	No distributional assumptions required
Interpretability	High (coefficients)	High for small trees, low for deep trees
Stability	High	Low (sensitive to data changes)
Overfitting risk	Moderate	High without regularization
Prediction smoothness	Smooth continuous function	Stepwise / piecewise constant
Best used when	Relationships are simple, explainability and causal interpretation matter	Relationships are complex, nonlinear, and prediction accuracy is the priority

12.9 Case Studies

Try the practice problems below to reinforce your understanding of decision tree regression modeling.

Case #1: Diamonds Dataset

This practice uses the **Diamonds** dataset that ships with the Seaborn Python package. Your goal is to build a **predictive** regression model using a **decision tree** to estimate diamond *price* as accurately as possible on new data. You will evaluate performance using *MAE*, *RMSE*, and R^2 on holdout data and explore how tree depth affects overfitting.

Dataset attribution: The Diamonds dataset is distributed with the Seaborn data repository and can be loaded with `seaborn.load_dataset("diamonds")`. If

you want the underlying CSV source, Seaborn hosts it in its public GitHub repository under *seaborn-data*.

To load the dataset, use this code:

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("diamonds")
df.head()
```

Prediction goal: Predict *price* using numeric features (*carat*, *depth*, *table*, *x*, *y*, *z*) and categorical features (*cut*, *color*, *clarity*). Use a predictive workflow with train/test splits, preprocessing in an sklearn pipeline, and tree-based modeling.

Tasks

- Inspect the dataset: rows/columns, data types, and summary statistics for *price*.
- Create *X* and *y* where *y* = *price* and *X* includes the predictors listed above. Document your chosen feature set.
- Split the data into *training* and *test* sets (80/20) using *random_state=42*.
- Build an sklearn preprocessing pipeline that scales numeric predictors (StandardScaler) and one-hot encodes categorical predictors (OneHotEncoder with *handle_unknown="ignore"*). Fit preprocessing only on the training data.
- Establish a baseline model by predicting the *training-set mean* price for all test observations. Report test *MAE*, *RMSE*, and *R*².
- Train a *DecisionTreeRegressor* using default hyperparameters inside your pipeline. Evaluate test-set *MAE*, *RMSE*, and *R*².

- Visualize the trained tree using `sklearn.tree.plot_tree` (limit depth to a readable value such as 3). Save the image for interpretation.
- Extract and plot impurity-based *feature importance scores* from the trained tree. Create a horizontal bar chart of feature importances.
- Train at least three additional trees with different values of *max_depth* (for example: 2, 5, and unrestricted). Record training and test-set *RMSE* for each model.
- Compare how model complexity affects overfitting by examining how training error and test error change as depth increases.

Analytical questions

1. How many rows and columns are in the Diamonds dataset?
2. What is the mean value of *price* in the dataset?
3. What are the baseline model's test-set *MAE*, *RMSE*, and R^2 ? (mean predictor)
4. What are the default decision tree model's test-set *MAE*, *RMSE*, and R^2 ?
5. Which three features had the highest impurity-based importance scores in your tree?
6. Include your tree visualization (depth ≤ 3). Which feature is used at the root split, and what threshold or category is applied?
7. How did test-set *RMSE* change as *max_depth* increased? Identify which depth performed best on the test set.
8. At what depth did the model begin to show signs of overfitting? Explain using training vs test error.
9. Short reflection (3–5 sentences): Why can a deeper tree reduce training error but increase test error? How does this relate to the bias–variance

tradeoff?



Diamonds Decision Tree Practice Answers

These answers assume the Diamonds dataset was loaded with `seaborn.load_dataset("diamonds")` and that you used an 80/20 train/test split with `random_state=42`. The workflow used a decision tree regression model and reported baseline (mean predictor) performance, default-ish decision tree performance, impurity-based feature importances aggregated to raw features, and a small max-depth sweep to check for overfitting.

1. The Diamonds dataset contains 53940 rows and 10 columns.
2. The mean value of *price* is 3932.7997.
3. (Baseline mean predictor) Predicting the training-set mean for every observation yielded test-set $MAE = 3020.5058$, $RMSE = 3987.2222$, and $R^2 = -0.0001$ (negative due to rounding and the baseline being essentially “no skill”).
4. (Default-ish decision tree) The decision tree regression model dramatically outperformed the mean baseline, achieving test-set $MAE = 360.1982$, $RMSE = 745.2457$, and $R^2 = 0.9651$.
5. (Top-3 impurity-based importances, aggregated to raw features) The three most important raw input features were *carat* (0.634406), *y* (0.252611), and *clarity* (0.064107). These dominate because early splits on size and quality create large reductions in squared error.
6. (Tree visualization) In the plotted tree (first three levels), the root split was $num_carat \leq 0.416$. The left branch (True) then split on $num_y \leq -0.176$, while the right branch (False) split on $num_y \leq 1.271$,

illustrating how trees quickly separate the data using the most informative size-related signals.

7. (Depth comparison) Across the tested depths, the lowest test RMSE occurred at $max_depth = 10$, with $test\ RMSE = 721.4509$ (and $test\ MAE = 392.7596$, $test\ R^2 = 0.9673$).
8. (Overfitting evidence) Overfitting begins to appear when training RMSE continues to decrease but test RMSE stops improving (or rises). In this sweep, that pattern first appears at $max_depth = 10$: training RMSE drops from 864.0486 (depth 8) to 682.5897 (depth 10), while test RMSE rises from 866.8421 (depth 8) to 721.4509 (depth 10). Even though depth 10 still had the best test RMSE in this particular sweep, the widening gap between train and test error is the warning sign that additional depth is increasingly “memorizing” the training data.
9. Overall interpretation: Compared to the mean baseline, the tree achieves far lower error because it can model strong nonlinear structure in diamond pricing (especially size effects). The depth sweep shows the bias–variance tradeoff in action: shallow trees underfit (higher RMSE), moderate depth improves generalization, and very deep trees begin to show a growing train–test gap that signals rising variance and overfitting risk.

Case #2: Red Wine Quality Dataset

This practice uses the **Red Wine Quality** dataset (*winequality-red.csv*). Your goal is to build and evaluate a **decision tree regression** model that predicts wine *quality* as accurately as possible on new data.

Dataset attribution: This dataset originates from the UCI Machine Learning Repository (Wine Quality Data Set) and was published by Cortez et al. in “Modeling wine preferences by data mining from physicochemical

properties” (Decision Support Systems, 2009). It contains physicochemical measurements of red wines along with a sensory quality score.

The red wine quality dataset is [available in the prior chapter](#) if you need to reload it.

Modeling focus: You are practicing *predictive inference with decision trees*. You will evaluate performance using holdout data, visualize the learned tree structure, examine impurity-based feature importance, and study how tree depth affects overfitting.

Tasks

- Inspect the dataset: rows, columns, data types, and summary statistics for *quality*.
- Create a label vector y using *quality* and a feature matrix X using all remaining numeric predictors.
- Split the data into training (60%), validation (20%), and test (20%) sets using fixed random seeds.
- Compute a baseline model that predicts the training-set mean of *quality* for every observation. Evaluate its MAE, RMSE, and R^2 on the test set.
- Fit a *DecisionTreeRegressor* using default hyperparameters. Evaluate its MAE, RMSE, and R^2 on the test set.
- Visualize the first 3 levels of the trained tree and identify the root split feature and threshold.
- Compute impurity-based feature importance values and create a horizontal bar chart of the top 10 features.
- Aggregate importance values to the raw feature level if preprocessing created multiple encoded columns.

- Train multiple trees using different values of *max_depth* (for example: 2, 3, 4, 5, 6, 8, 10). Record training and test RMSE for each.
- Create a line chart showing training RMSE and test RMSE versus *max_depth*.
- Identify the depth that produces the lowest test RMSE and treat this as your regularized model.
- Refit the model using the selected depth on the combined training + validation data and evaluate once on the test set.

Analytical questions

1. How many rows and columns are in the Red Wine Quality dataset?
2. What is the mean value of *quality*?
3. (Baseline) What are the test-set MAE, RMSE, and R^2 when predicting the training-set mean?
4. (Default tree) What are the test-set MAE, RMSE, and R^2 for the default decision tree model?
5. (Feature importance) What are the top three most important features according to impurity-based importance?
6. (Tree structure) Which feature appears at the root split, and what threshold is used?
7. (Depth comparison) Which *max_depth* value produced the lowest test RMSE?
8. (Overfitting) At what depth did training RMSE continue to decrease while test RMSE stopped improving or worsened?
9. (Final model) What are the test-set MAE, RMSE, and R^2 of the regularized model?

10. (Interpretation) In 2–3 sentences, explain why the regularized tree outperforms both the baseline and the unrestricted tree.



Red Wine Quality Decision Tree Practice Answers

These answers were computed by loading *winequality-red.csv* (1599 rows, 12 columns), using an 80/20 train/test split with *random_state=42*, and fitting a *DecisionTreeRegressor* to predict *quality*. The baseline model predicts the training-set mean for every test case. RMSE was computed using *rmse = np.sqrt(mean_squared_error(y_test, y_pred))*. A depth sweep compared multiple *max_depth* values, then the final model was refit on the combined train+validation data using the chosen depth and evaluated once on the untouched test set.

1. The Red Wine Quality dataset contains 1599 rows and 12 columns.
2. The mean value of *quality* is 5.6360.
3. (Baseline mean predictor on TEST) *MAE* = 0.6825, *RMSE* = 0.8092, and $R^2 = -0.0021$.
4. (Default-ish decision tree on TEST) *MAE* = 0.5094, *RMSE* = 0.8043, and $R^2 = 0.0101$.
5. (Root split from the tree visualization) The root split uses *alcohol* with threshold 11.55 (i.e., *alcohol* $\leq$ 11.55 at the root).
6. (Top-3 feature importances, impurity-based) *alcohol* = 0.260699, *sulphates* = 0.168462, and *volatile acidity* = 0.102135.
7. (Depth comparison) The lowest test RMSE in the depth sweep occurred at *max_depth* = 4, with *train RMSE* = 0.6140 and *test RMSE* = 0.6715.

8. (Overfitting evidence) Overfitting begins to appear after *max_depth* = 4: training RMSE continues to decrease (from 0.6140 at depth 4 to 0.5626 at depth 5), while test RMSE worsens (from 0.6715 at depth 4 to 0.6981 at depth 5) and eventually rises further (e.g., 0.7681 at depth 10).
9. (Final selected model on TEST) Using *max_depth* = 4 and refitting on train+validation, the final model achieved *MAE* = 0.5170, *RMSE* = 0.6618, and R^2 = 0.3298 on the test set.
10. (Interpretation) The tree outperforms the mean-baseline because it learns nonlinear threshold rules (for example, splitting first on *alcohol*) that create groups of wines with systematically different typical quality. The strongest impurity-based signals in this run come from *alcohol*, *sulphates*, and *volatile acidity*, so the model reduces error by partitioning the dataset along those chemistry-driven differences rather than predicting the same mean value for everyone.

Case #3: Bike Sharing Daily Dataset (Decision Tree Regression)

This practice uses the **Bike Sharing** daily dataset (*day.csv*). Your goal is to build a **decision tree regression** model that predicts total daily rentals (*cnt*) using a full predictive workflow: train/validation/test splits, preprocessing pipelines, baseline comparison, tree visualization, feature importance analysis, and overfitting control using tree depth.

Dataset attribution: This dataset is distributed as part of the Bike Sharing Dataset hosted by the UCI Machine Learning Repository (Fanaee-T and Gama). It contains daily rental counts and weather/context variables derived from the Capital Bikeshare system in Washington, D.C.

The bike sharing daily dataset is [available in the prior chapter](#) if you need to re-download it.

Important modeling note: Do not include *casual* or *registered* as predictors because they directly sum to *cnt* and would leak the answer into the model.

Goal: Build a predictive regression model that minimizes out-of-sample error (MAE and RMSE) using a decision tree, while controlling overfitting through depth regularization.

Tasks

- Inspect the dataset: number of rows, number of columns, and summary statistics for *cnt*.
- Define the label vector $y = cnt$ and the feature matrix X using the following raw predictors: *season*, *yr*, *mnth*, *holiday*, *workingday*, *weathersit*, *temp*, *atemp*, *hum*, and *windspeed*.
- Split the data into training (60%), validation (20%), and test (20%) sets using *random_state* = 42.
- Build a preprocessing pipeline that imputes missing values, standardizes numeric variables, and one-hot encodes categorical variables.
- Compute a baseline model that predicts the training-set mean of *cnt* for all observations and evaluate its test-set MAE, RMSE, and R^2 .
- Fit a baseline *DecisionTreeRegressor* using default hyperparameters and evaluate its test-set MAE, RMSE, and R^2 .
- Visualize the first three levels of the fitted decision tree and identify the root split feature and threshold.
- Compute impurity-based feature importance scores, aggregate them to raw features, and plot the top 10 features.
- Perform a depth sweep by training trees with multiple *max_depth* values (for example: 2, 3, 4, 5, 6, 8, 10) and record training and validation RMSE and MAE for each.

- Choose the depth that minimizes validation RMSE and refit the model using training + validation data.
- Evaluate the final selected model once on the untouched test set.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Bike Sharing daily dataset?
2. What is the mean value of *cnt*?
3. (Baseline) What are the test-set MAE, RMSE, and R^2 when predicting the training-set mean for every observation?
4. (Baseline tree) What are the test-set MAE, RMSE, and R^2 of the default decision tree model?
5. (Tree structure) What feature and threshold are used at the root node of your visualized tree?
6. (Feature importance) What are the top three raw features by impurity-based importance?
7. (Depth tuning) Which *max_depth* value produced the lowest validation RMSE?
8. (Final model) What are the test-set MAE, RMSE, and R^2 of the final tuned model?
9. (Interpretation) In 2–3 sentences, explain why the tuned tree outperforms both the mean baseline and the untuned tree.



Bike Sharing Decision Tree Practice Answers

These answers were computed using the Bike Sharing daily dataset (*day.csv*) with a 60/20/20 split (train/validation/test) using *random_state=42*. The label

was *cnt*, and the raw predictors were *season*, *yr*, *mnth*, *holiday*, *workingday*, *weathersit*, *temp*, *atemp*, *hum*, and *windspeed* (excluding *casual* and *registered* to avoid leakage).

1. The Bike Sharing daily dataset contains 731 rows and 16 columns.
2. The mean value of *cnt* is 4504.3488.
3. (Baseline) Predicting the training-set mean *cnt* for every observation yields test-set $MAE = 1718.0928$, $RMSE = 2030.5806$, and $R^2 = -0.0283$.
4. (Baseline tree) The DecisionTreeRegressor baseline achieved test-set $MAE = 842.4460$, $RMSE = 1108.7808$, and $R^2 = 0.6934$.
5. (Depth sweep) Validation RMSE was lowest at $max_depth = 6$ with $val RMSE = 841.2405$ (and $val MAE = 636.8526$), so $max_depth = 6$ is a reasonable stopping choice in this run.
6. (Final model) After refitting a tree with $max_depth = 6$ on the combined train+validation data and evaluating once on the untouched test set, the model achieved $MAE = 676.7794$, $RMSE = 946.6050$, and $R^2 = 0.7765$.
7. (Feature importance) The top three impurity-based feature importances (aggregated to raw features) were $temp = 0.448764$, $yr = 0.307456$, and $atemp = 0.107635$.
8. (Tree structure) The root split in the plotted tree was on *temp* with threshold -0.3738.
9. (Interpretation) The selected tree outperforms the mean-baseline because it can model strong nonlinear and interaction-driven patterns in rentals (especially temperature effects and the year-over-year trend captured by *yr*). Limiting depth to 6 helps avoid memorizing the

training data, improving generalization and lowering both average error (MAE) and large-error penalty (RMSE).

12.10 Assignment

Complete the assignment below:

This assessment can be taken [online](#).